UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

INTELIGENCIA ARTIFICIAL Y REDES NEURONALES

ACTIVIDAD 5: PROCESAMIENTO DE CONJUNTOS DE DATOS EN PYTHON

ING: DANIEL ISAIAS LOPEZ PAEZ

HORA: N5

Nombre	Matricula	Carrera
Agustin Azael Tamez Galaviz	2173925	IMTC
Diego Antonio Garcia Cardiel	2046605	IMTC
Zuriel Emiliano Jasso Davila	2177803	IMTC
Luis Angel Araujo Solis	1411762	IMTC
Juan Pablo Flores Perez	1903468	IMTC

Objetivo

- Familiarizarse con el lenguaje de programación Python y recordar conceptos de lógica de programación en general.

- Conocer la interfaz y funcionamiento de Google Colab.

- Usar librerías como Matplotlib, Pandas y Numpy.

Pasos De Proceso De Realizacion

En esta actividad, trabajé con un conjunto de datos (robot_sensors.csv) que contenía mediciones de sensores de un robot móvil. El objetivo fue limpiar los datos, normalizarlos y visualizarlos para su posterior uso en aplicaciones de inteligencia artificial y redes neuronales. Para ello, seguí un enfoque estructurado utilizando Python en un notebook de Google Colab, con librerías como Pandas, NumPy, Scikit-learn y Matplotlib.

1. **Cargar el dataset con pandas**

Lo primero que hice fue cargar el dataset en Google Colab y leerlo con Pandas para analizar su estructura. Después subí el archivo robot_sensors.csv a la sesión de Google Colab y utilicé pandas.read_csv () para cargar los datos en un DataFrame.

```
import pandas as pd

# Cargar el dataset
df = pd.read_csv('robot_sensors.csv')
```

2. Mostrar primeras filas del dataset

Para entender el contenido del dataset, revisé las primeras cinco filas con df.head(). Esto me permitió ver las columnas disponibles y sus valores.

```
print(df.head())
```

Después usé df.info() para revisar el tipo de datos en cada columna y detectar valores nulos.

```
print(df.info())
```

También utilicé df.describe() para obtener estadísticas generales como los valores mínimos, máximos y promedios de cada columna.

```
print(df.describe())
```

3. Limpiar los datos

Identifiqué las filas con valores nulos y las eliminé con dropna(), asegurando que solo se trabajara con datos completos.

Eliminación de valores nulos

```
df = df.dropna()
```

Eliminación de valores atípicos

Al analizar los datos, noté que algunos valores no tenían sentido, como velocidades negativas, lo cual es ilógico en el contexto del movimiento del robot. Para corregir esto, eliminé las filas donde la velocidad fuera menor a cero.

```
df = df[df['Velocidad (cm/s)'] >= 0]
```

4. Normalizar los datos numéricos con Min-Max Scaling

Para garantizar que todas las variables estuvieran en la misma escala y evitar que unas tuvieran más peso que otras en futuros modelos de inteligencia artificial, normalicé los datos numéricos utilizando **Min-Max Scaling** de la librería sklearn.preprocessing.

El **Min-Max Scaling** transforma los datos para que estén en un rango de **0 a 1**, preservando la distribución original, pero haciéndolos comparables.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df[['Distancia (cm)', 'Velocidad (cm/s)', 'Temperatura (°C)']] = scaler.fit_transform(df[[
```

```
']] = scaler.fit_transform(df[['Distancia (cm)', 'Velocidad (cm/s)', 'Temperatura (°C)']])
```

5. **Visualizar los datos normalizados usando matplotlib (valores vs tiempo)**

Para analizar la evolución de los valores de los sensores a lo largo del tiempo, generé gráficos utilizando Matplotlib. Y representé los valores normalizados de distancia, velocidad y temperatura en función del tiempo para observar sus tendencias y comportamientos.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.plot(df['Tiempo (s)'], df['Distancia (cm)'], label='Distancia (cm)')
plt.plot(df['Tiempo (s)'], df['Velocidad (cm/s)'], label='Velocidad (cm/s)')
plt.plot(df['Tiempo (s)'], df['Temperatura (°C)'], label='Temperatura (°C)')

plt.xlabel('Tiempo (s)')
plt.ylabel('Valores Normalizados')
plt.title('Evolución de los sensores del robot')
plt.legend()
plt.show()
```

Capturas De Códigos Y Graficas

Procesamiento de conjuntos de datos en Python

```
[ ] import numpy as np
    import matplotlib.pyplot as plt
    import pandas as pd
```

```
[ ] from google.colab import files
```

```
[ ] from google.colab import drive
    drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
[ ] import pandas as pd
    df = pd.read_csv("/content/drive/MyDrive/Bases_Datos google_Colab/Actividad_5/robot_sensors.csv")
    df.shape
```

```
(1000, 4)
```

```
[ ] # El tamaño de los datos

    print("\nEl tamaño de los datos por filas y columnas",df.shape)
```

```
El tamaño de los datos por filas y columnas (1000, 4)
```

```
df.head(9)
```

	Tiempo (s)	Distancia (cm)	Velocidad (cm/s)	Temperatura (°C)
0	0	43.708611	9.256646	23.925585
1	1	95.564288	27.095047	23.704682
2	2	75.879455	43.647292	33.593819
3	3	63.879264	36.611244	23.743193
4	4	24.041678	40.328057	24.079246
5	5	24.039507	32.939168	31.390974
6	6	15.227525	34.613828	26.746098
7	7	87.955853	42.459783	31.650658
8	8	64.100351	12.483400	20.980492

Próximos pasos: Generar código con df | Ver gráficos recomendados | New interactive sheet

```
[ ] df_filtered=df.dropna()
    df_filtered.shape
    #aqui se estan imprimiendo los datos despues de eliminar las
    #celdas que tenian un 0
```

```
(913, 4)
```

```
[ ]  condition = (df_filtered < 0).any(axis=1)
    df_filtered = df_filtered.drop(df_filtered[condition].index)
    # y aqui se eliminaron las que tenian valores negativos auqne al

    print(df_filtered.shape)
```

```
(913, 4)
```

Ahora debemos hacer la normalizacion de la informacion, en este caso resulta util realizar una normalizacion ya que los sensores detectan variaciones muy debiles por lo cual al normalizar elimina valores por asi decirlos insignificantes y al ser varibles medidos en distintas unidades al normalizarlas lleva a magnitudes a valores equiparables, para que la grafica no salga tan rara

```
[ ] !pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.14.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.6.0)
```

```
[ ] from sklearn.preprocessing import StandardScaler

    scaler = StandardScaler()
    df_scaled = scaler.fit_transform(df_filtered)
    df_scaled = pd.DataFrame(df_scaled, columns=df_filtered.columns, index=df_filtered.index)

    print(df_scaled.head())
```

```
   Tiempo (s)  Distancia (cm)  Velocidad (cm/s)  Temperatura (°C)
0   -1.721106       -0.392706         -1.099361         -0.825923
1   -1.717651        1.570831          0.121176         -0.876415
2   -1.714196        0.825457          1.253712          1.383975
3   -1.710741        0.371064          0.772292         -0.867613
4   -1.707287       -1.137403          1.026604         -0.790800
```

Varios valores salieron negativos porque la funcion " scaler = StandardScaler() " reorganiza los valores escalandolos desde el 0 generando 2 histogramas un decreciente de 0 a (-) y la otra de 0 a (+)

```
[ ] print(df_filtered.columns)
    #aqui verificamos el nombre de las columnas para poder hacer sus graficas
```

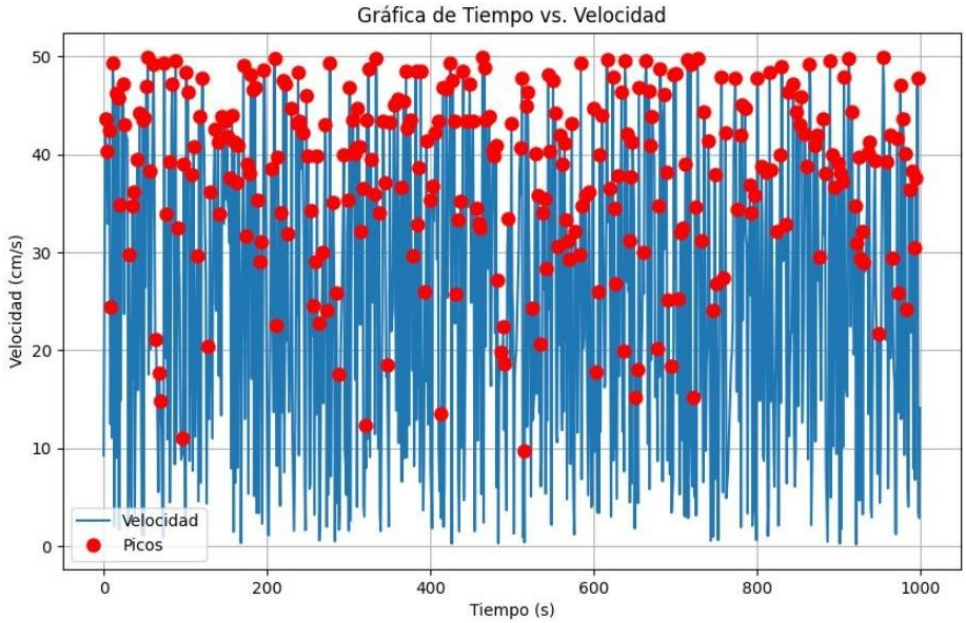
```
Index(['Tiempo (s)', 'Distancia (cm)', 'Velocidad (cm/s)', 'Temperatura (°C)'], dtype='object')
```

```
[ ] import matplotlib.pyplot as plt
    import numpy as np

    # ... (tu código anterior para cargar y filtrar datos) ...

    # Calcular los picos (crestas)
    x = df_filtered['Tiempo (s)'].values
    y = df_filtered['Velocidad (cm/s)'].values
    picos_indices = np.where((y[1:-1] > y[:-2]) & (y[1:-1] > y[2:]))[0] + 1  # Indices de los picos
    picos_x = x[picos_indices]
    picos_y = y[picos_indices]

    # Graficar
    plt.figure(figsize=(10, 6))
    plt.plot(df_filtered['Tiempo (s)'], df_filtered['Velocidad (cm/s)'], label='Velocidad')  # Agregar label para la leyenda
    plt.plot(picos_x, picos_y, 'ro', markersize=8, label='Picos')  # Marcar los picos con puntos rojos
    plt.title('Gráfica de Tiempo vs. Velocidad')
    plt.xlabel('Tiempo (s)')
    plt.ylabel('Velocidad (cm/s)')
    plt.grid(True)
    plt.legend()  # Mostrar la leyenda
    plt.show()
```



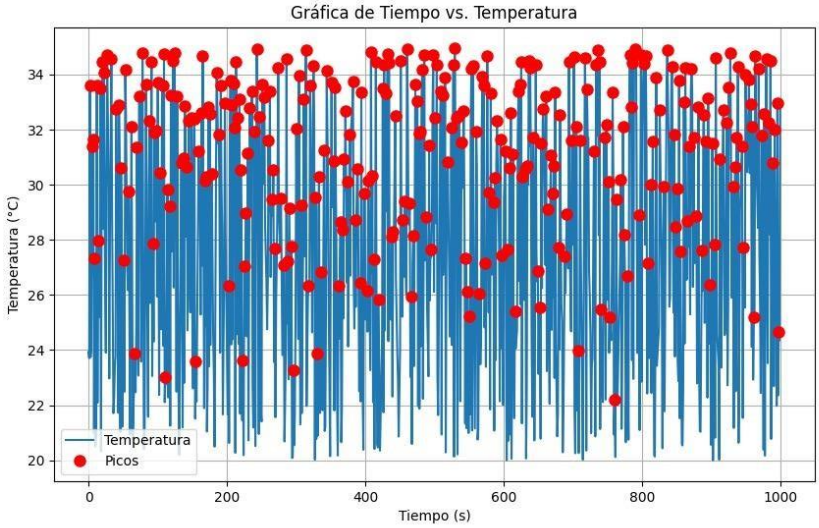

```
import matplotlib.pyplot as plt
import numpy as np
```

```
import matplotlib.pyplot as plt
import numpy as np

# ... (tu código anterior para cargar y filtrar datos) ...

# Calcular los picos (crestas) - Ahora para Temperatura
x = df_filtered['Tiempo (s)'].values
y = df_filtered['Temperatura (°C)'].values  # Cambiamos 'Velocidad (cm/s)' por 'Temperatura (°C)'
picos_indices = np.where((y[1:-1] > y[:-2]) & (y[1:-1] > y[2:]))[0] + 1  # Indices de los picos
picos_x = x[picos_indices]
picos_y = y[picos_indices]

# Graficar
plt.figure(figsize=(10, 6))
plt.plot(df_filtered['Tiempo (s)'], df_filtered['Temperatura (°C)'], label='Temperatura')  # Cambiamos 'Velocidad (cm/s)' por 'Temperatura (°C)'
plt.plot(picos_x, picos_y, 'ro', markersize=8, label='Picos')  # Marcar los picos con puntos rojos
plt.title('Gráfica de Tiempo vs. Temperatura')
plt.xlabel('Tiempo (s)')
plt.ylabel('Temperatura (°C)')  # Cambiamos la etiqueta del eje y
plt.grid(True)
plt.legend()  # Mostrar la leyenda
plt.show()
```

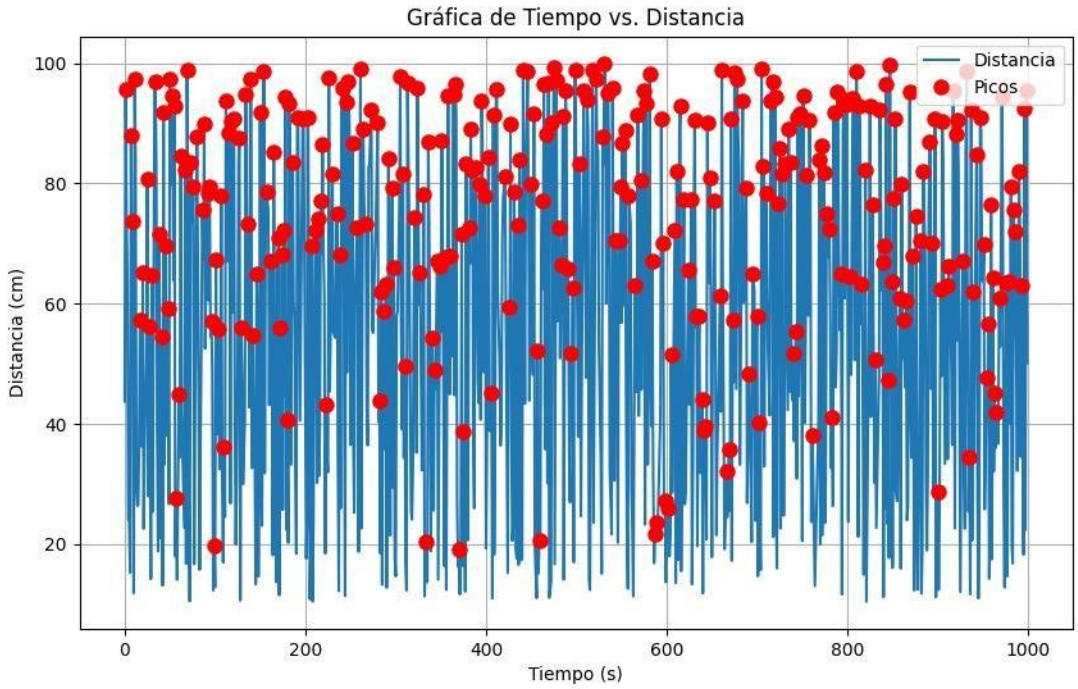


```
import matplotlib.pyplot as plt
import numpy as np

# ... (your previous code to load and filter data) ...

# Calculate the peaks (crests) - Now for Distance
x = df_filtered['Tiempo (s)'].values
y = df_filtered['Distancia (cm)'].values  # Changed to 'Distancia (cm)'
picos_indices = np.where((y[1:-1] > y[:-2]) & (y[1:-1] > y[2:]))[0] + 1  # Indices of the peaks
picos_x = x[picos_indices]
picos_y = y[picos_indices]

# Plot
plt.figure(figsize=(10, 6))
plt.plot(df_filtered['Tiempo (s)'], df_filtered['Distancia (cm)'], label='Distancia')  # Changed to 'Distancia (cm)'
plt.plot(picos_x, picos_y, 'ro', markersize=8, label='Picos')  # Mark peaks with red dots
plt.title('Gráfica de Tiempo vs. Distancia')
plt.xlabel('Tiempo (s)')
plt.ylabel('Distancia (cm)')  # Changed y-axis label
plt.grid(True)
plt.legend()  # Show legend
plt.show()
```



Preguntas

1. **¿Cómo afecta la limpieza de datos al análisis del dataset?**

La limpieza de datos es crucial para mejorar la calidad del análisis y la precisión de los modelos de inteligencia artificial. Un dataset con datos inconsistentes, duplicados o con valores faltantes puede llevar a interpretaciones erróneas y modelos ineficientes. La limpieza de datos ayuda a reducir el ruido, eliminar sesgos y mejorar la confiabilidad de los resultados, asegurando que el modelo trabaje con información representativa y útil.

2. **¿Por qué es importante la normalización en Machine Learning?**

La normalización es importante porque escala los datos dentro de un rango específico, evitando que características con valores numéricos grandes dominen sobre otras con valores más pequeños. Esto es especialmente útil en algoritmos que dependen de la distancia entre puntos de datos, como k-NN o redes neuronales, ya que mejora la convergencia del modelo y su estabilidad al entrenarlo.

3. **¿Cómo interpretarías los datos visualizados?**

La interpretación de los datos visualizados depende del tipo de gráfico utilizado. En general, se debe analizar la distribución de los datos, identificar patrones, tendencias, valores atípicos y correlaciones entre variables. Por ejemplo, un histograma permite observar la frecuencia de los datos, mientras que un diagrama de dispersión puede revelar relaciones entre variables. La correcta interpretación facilita la toma de decisiones y la comprensión del comportamiento del dataset.

Conclusion

El desarrollo de esta actividad permitió aplicar conceptos fundamentales en el preprocesamiento de datos, una fase crucial en cualquier proyecto de inteligencia artificial y redes neuronales. Durante el proceso, se llevaron a cabo varias tareas esenciales, como la exploración inicial de los datos, la limpieza de valores nulos y atípicos, la normalización de las variables y la visualización gráfica para interpretar mejor los resultados. Uno de los aspectos más importantes fue la eliminación de valores inconsistentes. La presencia de valores nulos puede generar errores en modelos de aprendizaje automático, ya que estos algoritmos requieren datos completos para realizar predicciones precisas. Del mismo modo, los valores atípicos, como velocidades negativas en un robot en movimiento, pueden distorsionar el análisis y afectar el desempeño de los modelos de inteligencia artificial. Al eliminar estos valores, garantizamos que el dataset sea más fiable y representativo de la realidad. La normalización de los datos también fue un paso fundamental, ya que permitió escalar las variables numéricas a un mismo rango, facilitando su interpretación y asegurando que todas tengan la misma importancia dentro de los modelos de inteligencia artificial.

Referencias

- Matthes, E. (2023). Python crash course. No starch press
- https://www.w3schools.com/python/default.asp

Link De La Actividad

https://github.com/agustintamez/Actividades/blob/main/AF5_Procesamiento_De_Datos.ipynb

.